Logické funkce, Booleova algebra, logické výrazy. Způsoby vyjadřování logických funkcí. Kombinační a sekvenční logické obvody. Bistabilní paměťové členy: SR, JK, T, D

HEX

Logické funkce

logická proměnná – obecně nezávislá matematická proměnná, která může nabývat konečného počtu hodnot – dvouhodnotová logika – logická 1 nebo logická 0

logická funkce – představuje soubor pravidel pro jednoznačné přiřazení hodnot závisle proměnných (funkčních hodnot) k jednotlivým kombinacím nezávislých proměnných (logických proměnných) - logické funkce obvykle zapisujeme písmenem f s indexem nebo velkými písmeny - pro n proměnných máme 2n různých kombinací a 22n různých logických funkcí - určitá funkce – jednoznačně určena hodnota pro všech 2n kombinací proměnných - neurčitá funkce – pro alespoň jednu kombinaci proměnných není určena funkční hodnota - tím pádem logická funkce může nabývat – logická 1, logická 0, X – neurčitý stav - v jazyce VHDL definován datový typ std_logic – 9 hodnot (viz VHDL přednášky)

pravdivostní tabulka – tabulkový výčet kombinací vstupních proměnných a jim odpovídající funkční hodnota – úplná či neúplná (viz dále)

úplný soubor funkcí – soubor (skupina) elementárních logických funkcí, pomocí nichž lze vyjádřit libovolnou logickou funkci

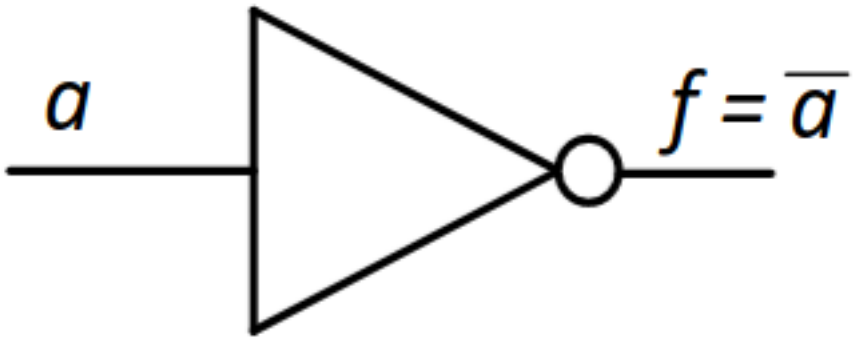
minimální úplný soubor funkcí – minimální soubor elementárních logických funkcí, pomocí nichž lze vyjádřit libovolnou logickou funkci

funkce negace (NOT)

nabývá vždy přesně opačné hodnoty než proměnná a

a	$f(a)$
0	1
1	0

$$f(a) = \overline{a}; \quad f(a) = \neg a; \quad f(a) = \text{NOT } a$$

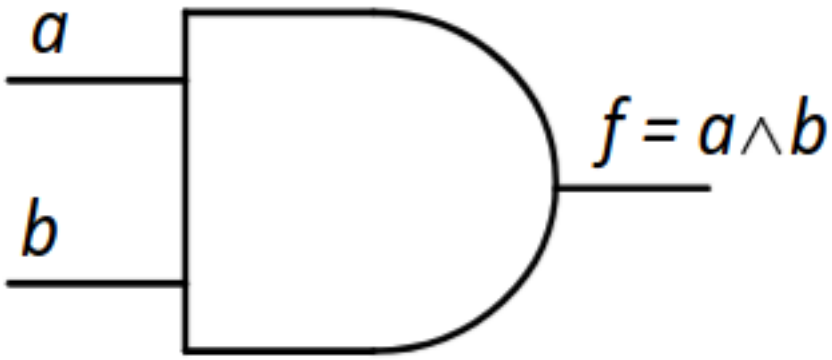


logický součin (AND)

hodnota funkce je logická 1 pouze pokud obě vstupní proměnné a, b nabývají hodnoty logická 1, jinak je její hodnota logická 0

a	b	$f(a \cdot b)$
0	0	0
0	1	0
1	0	0
1	1	1

$$f(a \cdot b) = a \wedge b = a \cdot b = ab = a \text{ AND } b$$

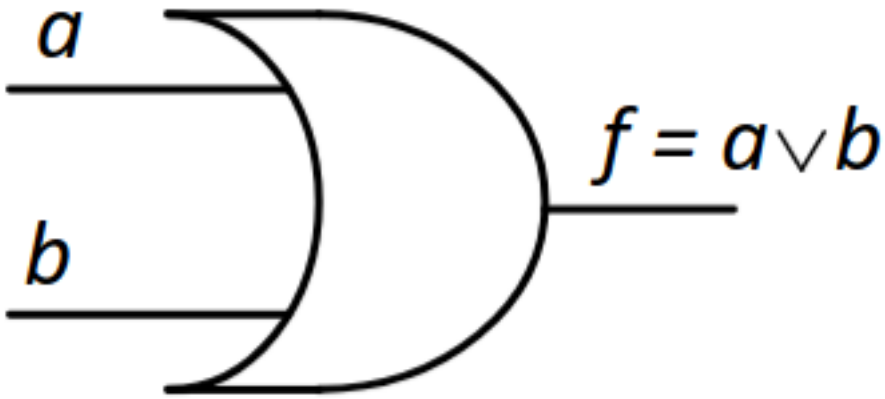


logický součet (OR)

funkce nabývá logické 0 pouze když obě proměnné a, b jsou logické 0, pokud alespoň jedna z nich má hodnotu logická 1, hodnota funkce je logická 1

a	b	$f(a+b)$
0	0	0
0	1	1
1	0	1
1	1	1

$$f(a+b) = a \vee b = a + b = a \text{ OR } b$$



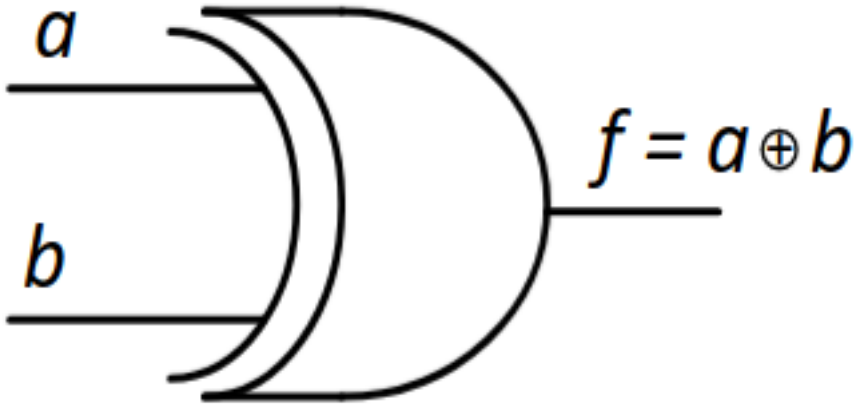
exclusive OR (XOR)

tato funkce nabývá hodnoty logická 1 pokud hodnoty obou proměnných jsou navzájem různé, a hodnoty logická 0 pokud jsou stejné

podobná funkci logického součtu, ale na rozdíl od něj dává funkce logickou 0 pokud a = b = 1 (logický součet dává logickou 1 pro tuto vstupní kombinaci)

a	b	$f(a \oplus b)$
0	0	0
0	1	1
1	0	1
1	1	0

$$f(a \oplus b) = a \oplus b = \overline{a} \cdot b \vee a \cdot \overline{b}$$



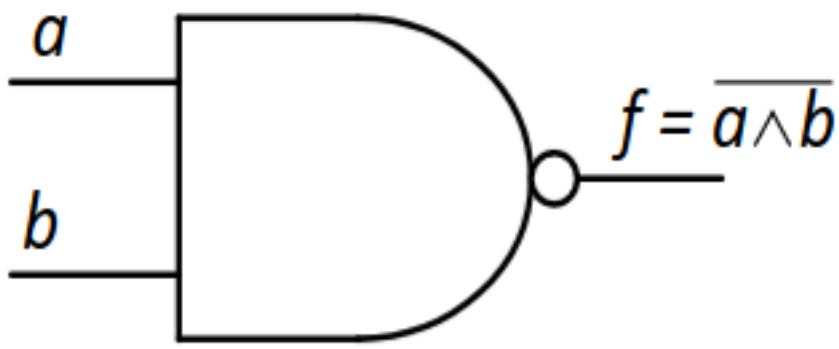
negace logického součinu (NAND)

funkce negace logického součinu, hodnota funkce je logická 0 pokud obě proměnné a, b nabývají hodnoty logická 1, v opačném případě je výstup logická 1

přesně opačná funkce k funkci logický součin – negace logického součinu, v literatuře také nazývána Shefferova funkce

a	b	$f(\overline{a \cdot b})$
0	0	1
0	1	1
1	0	1
1	1	0

$$f(\overline{a \cdot b}) = \overline{a \wedge b} = \overline{a \cdot b} = \overline{ab} = a \text{ NAND } b$$



pozor na správný zápis negace, negace logického součinu a a b není rovna logickému součinu negace a a negace b!

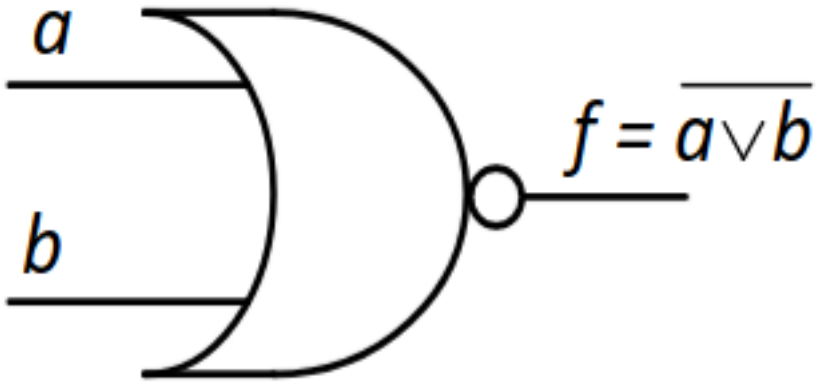
negace logického součtu (NOR)

opačná funkce k funkci logický součet, její funkční hodnota je logická 1 pouze když obě proměnné a, b jsou logické 0, jinak je její hodnota logická 0

je to negace logického součtu – NOT OR = NOR, v literatuře též Piercova funkce

a	b	$f(\overline{a+b})$
0	0	1
0	1	0
1	0	0
1	1	0

$$f(\overline{a+b}) = \overline{a \vee b} = \overline{a+b} = a \text{ NOR } b$$



pozor na negaci, negace logického součtu a a b není rovna logickému součtu negace a a negace b!

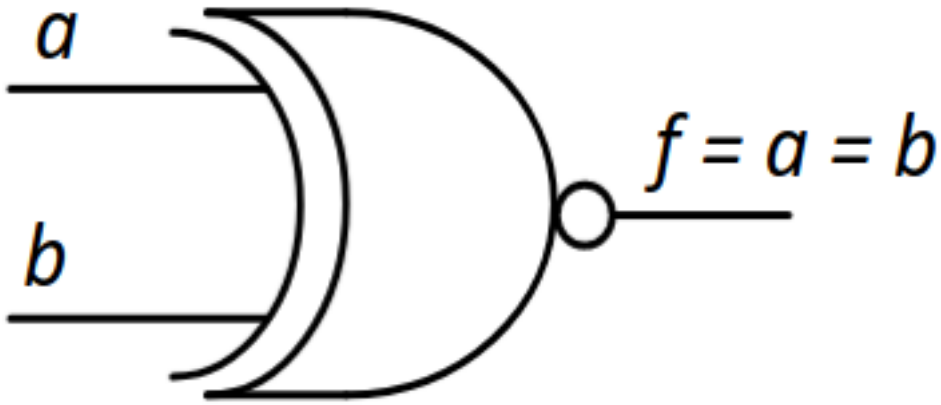
XNOR

opačná funkce k funkci XOR, její hodnota je logická 1 pokud hodnoty obou vstupních proměnných a, b jsou shodné, pokud a, b jsou různé, dává logickou 0

negace XOR = XNOR, někdy též funkce ekvivalence

a	b	$f(\overline{a \oplus b})$
0	0	1
0	1	0
1	0	0
1	1	1

$$f(\overline{a \oplus b}) = \overline{a \oplus b} = a \equiv b = \overline{a} \cdot \overline{b} \vee a \cdot b$$



Booleova algebra

algebra = obecně matematický prostor obsahující definice symbolů, operací a pravidel pro manipulaci s těmito symboly

Booleova algebra – část algebry zabývající se logickými proměnnými a funkcemi

- definovány dva základní prvky – logická 0 (nepravda), logická 1 (pravda)
- Booleova algebra založena na úplném souboru funkcí – AND, OR, NOT

Name	AND form	OR form
Identity law	$1A = A$	$0 + A = A$
Null law	$0A = 0$	$1 + A = 1$
Idempotent law	$AA = A$	$A + A = A$
Inverse law	$A\overline{A} = 0$	$A + \overline{A} = 1$
Commutative law	$AB = BA$	$A + B = B + A$
Associative law	$(AB)C = A(BC)$	$(A + B) + C = A + (B + C)$
Distributive law	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption law	$A(A + B) = A$	$A + AB = A$
De Morgan's law	$\overline{AB} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A}\,\overline{B}$

v Booleově algebře má logický součin vyšší prioritu než logický součet

Logické výrazy

Logický výraz je *modelem chování* logického systému – hodnoty logické funkce získáme jeho vyhodnocením pro jednotlivé hodnoty vstupních proměnných. Kromě toho může být také *modelem struktury* logického systému – pomocí souboru kontaktních či bezkontaktních prvků lze podle něj sestavit logický obvod

Zápis logického výrazu obsahuje – kromě logických proměnných a funkce – symboly logických spojek. Větné spojce „*a současně*“ (*logickému součinu*) odpovídá symbol ·, spojce „*nebo*“ (*logickému součtu*) symbol „+“ a tvrzení „*není pravda, že*“ (*logická negace*) se značí pruhem nad příslušnou proměnnou.

https://eluc.ikap.cz/verejne/lekce/932

Způsoby vyjadřování logických funkcí

1. Pravdivostní tabulka
2. Zkrácený seznam stavových indexů
3. Algebraický zápis
4. Mapy

Pravdivostní tabulka

tabulkový výčet jednotlivých kombinací nezávislých logických proměnných a jim přiřazených hodnot dané logické funkce

obvykle přidáváme sloupec stavových indexů N

úplná pravdivostní tabulka – obsahuje všechny kombinace vstupních proměnných pro n vstupních proměnných obsahuje 2n řádků

neúplná (zkrácená) pravdivostní tabulka – z úplné pravdivostní tabulky vynecháme neurčité stavy (neurčité stavy zapisujeme X), počet řádků neúplné tabulky $\leq$ 2n

pokud dvě logické funkce mají navzájem shodné pravdivostní tabulky, jedná se o identické logické funkce (a naopak)

pravdivostní tabulka je vhodná pro popis logického obvodu (výstupní funkce), ale nehodí se pro minimalizaci ani realizaci pomocí hradel

úplná pravdivostní tabulka funkce *f*

N	c	b	a	f
0	0	0	0	0
1	0	0	1	×
2	0	1	0	1
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

zkrácená pravd. tabulka funkce *f*

N	c	b	a	f
0	0	0	0	0
2	0	1	0	1
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

s počtem proměnných n narůstá exponenciálně počet řádků úplné tabulky (2^n)

Zkrácený seznam stavových indexů

zkrácený zápis pravdivostní tabulky

seznam indexů N, ve kterých nabývá daná funkce logické 1 nebo neurčitého stavu pro rozlišení zapisujeme neurčité stavy do závorek

předchozí funkci f můžeme zapsat ve formě zkráceného seznamu stavových indexů:

$$f = (1), 2, 3, 5, 6, 7$$

Algebraický zápis

zápis ve formě funkčního vyjádření pomocí trojice elementárních logických funkcí – log. součtu, log. součinu a negace a s použitím pravidel a zákonů Booleovy algebry

- **nezávislá proměnná** – jedna vstupní proměnná funkce v přímém či negovaném tvaru
- **podstatný bod** – stav, kdy daná funkce nabývá hodnoty logická 1
- **nepodstatný bod** – stav, kdy funkce nabývá logické 0 nebo neurčitého stavu
- **term** – jeden výraz obsahující každou nezávislou proměnnou přesně jedenkrát
- **součinový term** – term obsahující jen operaci logického součinu – vytvořený jako logický součin vstupních nezávislých proměnných
- **součtový term** – obdobně, term obsahující jen operaci logický součet
- **minterm** – součinový term obsahující všechny vstupní nezávislé proměnné právě jednou v přímém či negovaném tvaru
- **maxterm** – obdobně, součtový term obsahující všechny proměnné právě jednou
- **úplná normální disjunktní forma (ÚNDF)** – forma funkce obsahující logické součty mintermů – součet součinů – v literatuře též Sum of Products, SoP - obsahuje tolik součinů, kolik má daná funkce podstatných (jednotkových) bodů
- **minimální normální disjunktní forma (MNDF)** – minimalizovaná forma ÚNDF dané funkce f, obsahující nejmenší možný počet mintermů

vyjádření funkce pomocí ÚNDF

1. pro každý řádek úplné pravdivostní tabulky dané funkce vytvoříme logický součin funkční hodnoty fN a vstupních proměnných a, b, c – pokud obsahuje pro daný řádek tabulky proměnná log. 1, zapíšeme ji přímo, v případě log. 0 zapíšeme její negaci
2. využijeme zákony o neutrálnosti jedničky a agresivity nuly
3. výslednou ÚNDF formu zapíšeme jako součet jednotlivých mintermů

$$f_D = 0 \cdot \overline{abc} \vee 1 \cdot a\overline{bc} \vee 1 \cdot \overline{a}b\overline{c} \vee 1 \cdot \overline{ab}c \vee 1 \cdot ab\overline{c} \vee 0 \cdot \overline{abc} \vee 1 \cdot a\overline{bc} \vee 1 \cdot \overline{a}bc \vee 1 \cdot abc$$

$$f_D = abc \vee ab\overline{c} \vee a\overline{b}c \vee a\overline{bc} \vee \overline{a}bc \vee \overline{abc}$$

převod ÚNDF - ÚNKF

konjunktní formu funkce získáme z těch řádků pravdivostní tabulky, kdy funkce f dává hodnotu logická 0 nebo neurčitý stav – zákon o neutrálnosti nuly

disjunktní formu funkce získáme z těch řádků pravdivostní tabulky, kdy funkce f dává hodnotu logická 1 nebo neurčitý stav – zákon o neutrálnosti jedničky

každou funkci f proto můžeme vždy zapsat v její disjunktní i konjunktní formě

1. nahradíme všechny logické součty ($\vee$) pomocí logických součinů ($\wedge$) a naopak
2. ve všech výrazech nahradíme jednotlivé proměnné jejich negacemi a naopak
3. musíme vždy převádět úplné formy, tzn. obsahující všechny řádky pravdivostní tabulky funkce (s hodnotou logická 0 i 1), neurčité stavy ponecháme vždy v obou formách

ÚNDF:

$$f_D = 0 \cdot \overline{abc} \vee X \cdot \overline{a}bc \vee 1 \cdot a\overline{bc} \vee 1 \cdot \overline{a}bc \vee 0 \cdot \overline{abc} \vee 1 \cdot a\overline{bc} \vee 1 \cdot \overline{a}bc \vee 1 \cdot abc =$$
$$= \overline{abc} \vee \overline{a}bc \vee a\overline{bc} \vee \overline{a}bc \vee \overline{abc} \vee abc$$

ÚNKF:

$$f_K = (0 \vee a \vee b \vee c) \cdot (X \vee \overline{a} \vee b \vee c) \cdot (1 \vee a \vee \overline{b} \vee c) \cdot (1 \vee a \vee b \vee \overline{c}) \cdot (0 \vee a \vee b \vee \overline{c}) \cdot (1 \vee \overline{a} \vee b \vee \overline{c}) =$$

$$= (a \vee b \vee c) \cdot (\overline{a} \vee b \vee c) \cdot (a \vee b \vee \overline{c})$$

Karnaughovy mapy

existuje více druhů map, Karnaughovy mapy do 6 proměnných nejvhodnější

je to grafická reprezentace funkce f

mapa má vždy 2n políček – kde n je počet nezávislých proměnných funkce - každé políčko v Karnaughově mapě představuje jeden řádek pravdivostní tabulky - každé políčko v Karnaughově mapě představuje jeden minterm funkce f

políčka v Karnaughově mapě očíslována pomocí Grayova kódu – indexy navzájem sousedních políček vyjádřené ve dvojkové soustavě se liší jen na jedné řádové pozici

proměnné v mapě obvykle přiřazujeme od a – LSB až „poslední“ – MSB

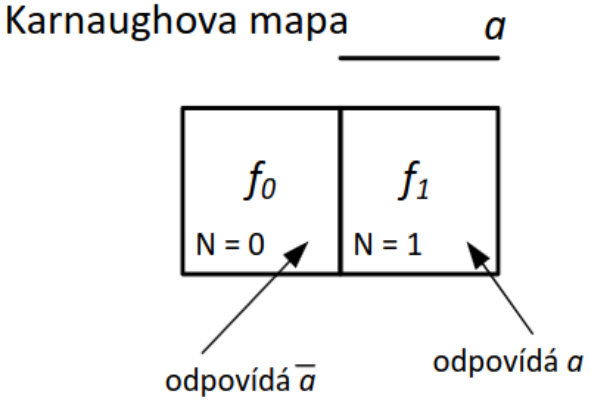
oblasti pokrytí mapy jednotlivými proměnnými znázorňujeme po stranách mapy

Karnaughovy mapy užitečné hlavně pro minimalizaci tvaru funkce

Karnaughova mapa 1 proměnné

pravdivostní tabulka

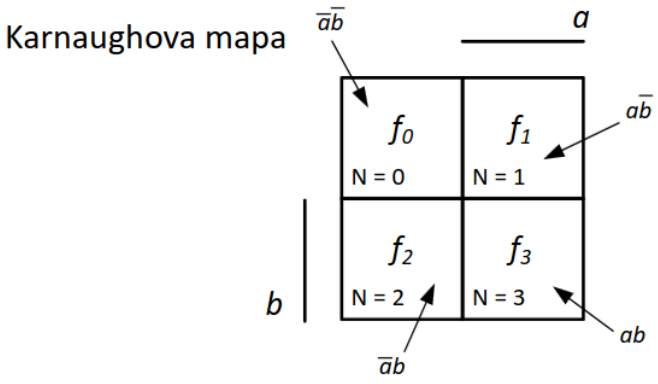
N	a	f
0	0	f_0
1	1	f_1



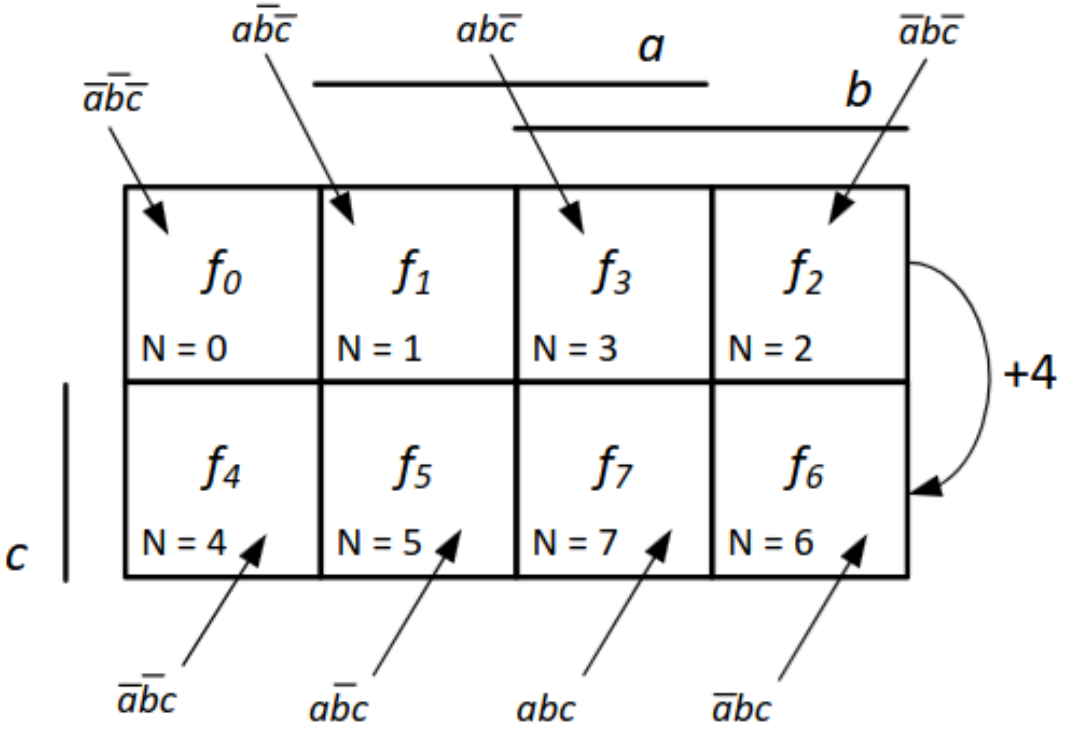
Karnaughova mapa 2 proměnných

pravdivostní tabulka

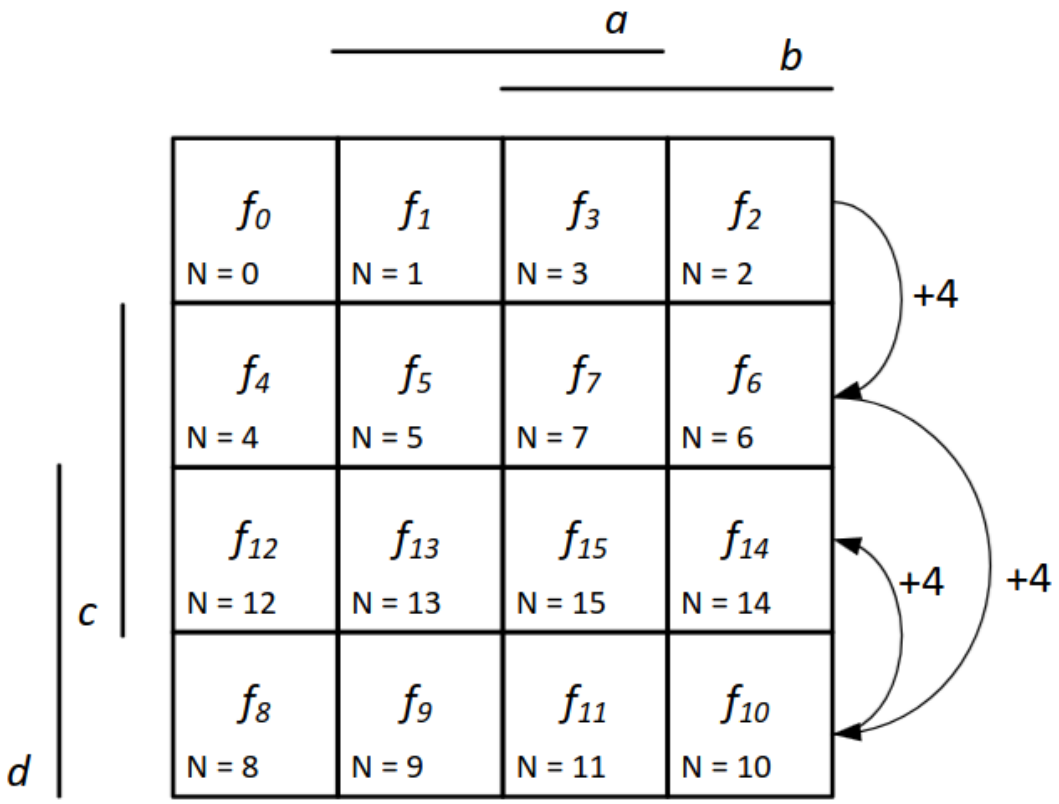
N	b	a	f
0	0	0	f_0
1	0	1	f_1
2	1	0	f_2
3	1	1	f_3



Karnaughova mapa 3 proměnných

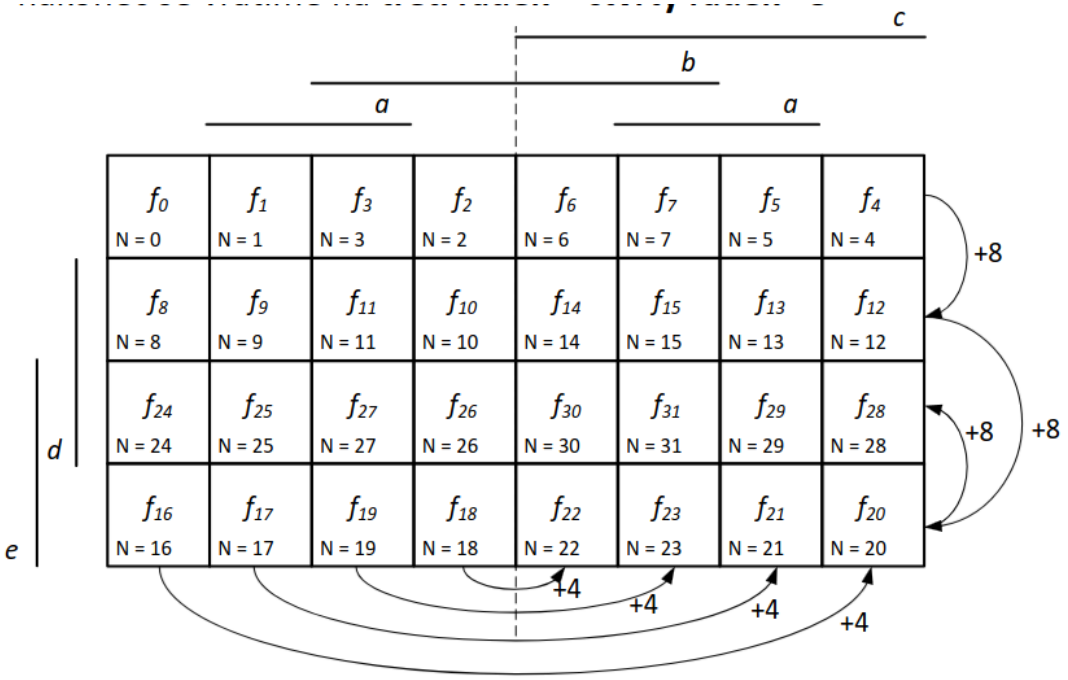


Karnaughova mapa 4 proměnných



Karnaughova mapa 5 proměnných

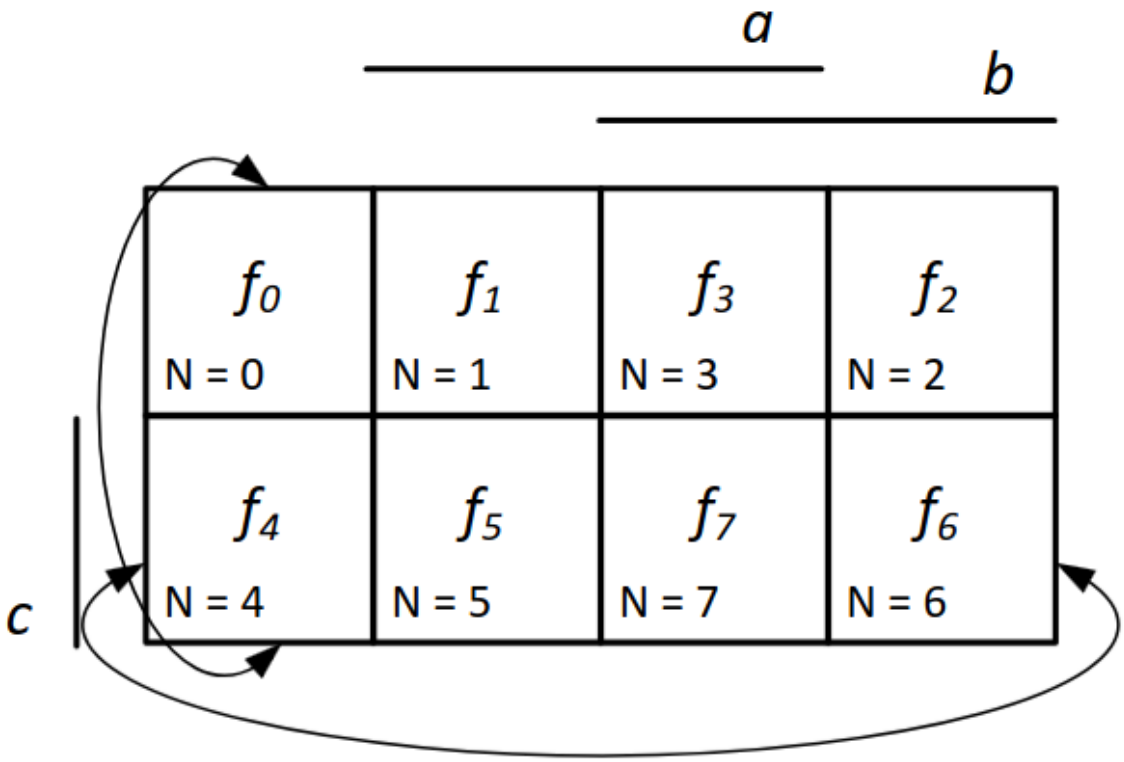
osa symetrie rozděluje mapu na dvě poloviny – levá + pravá část (osově symetrické)



Karnaughovy mapy – sousední políčka

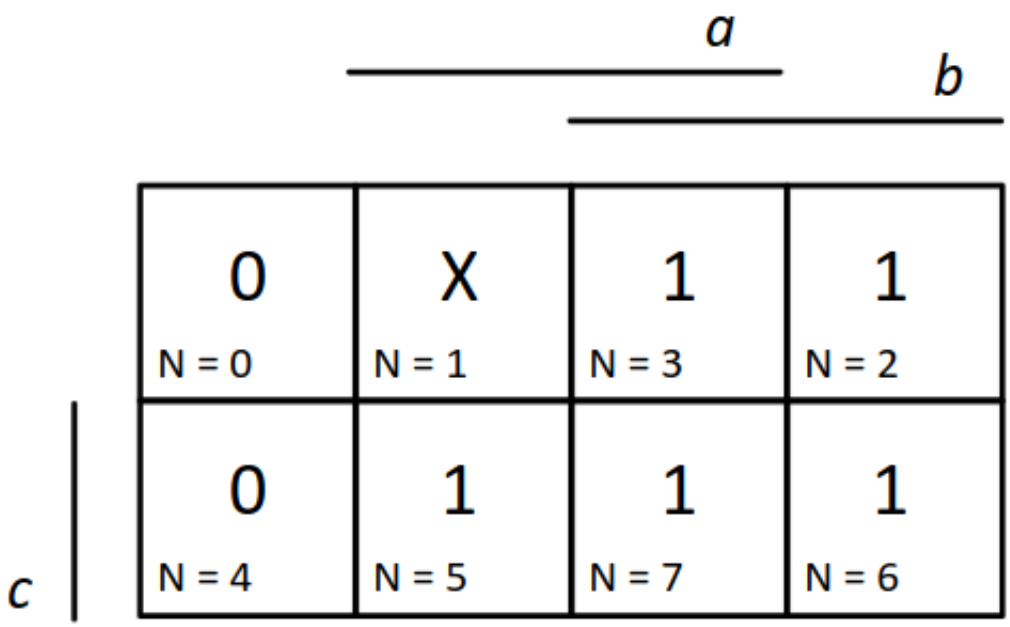
indexy sousedních políček v Karnaughově mapě zapsané ve dvojkové soustavě – změna jen na jedné řádové pozici – vždy jen jedna proměnná se liší (přímý vs. negovaný tvar)

každá Karnaughova mapa může být přeložena zleva-doprava, seshora-dolů, mapy pro 5 a více proměnných navíc osy symetrie – políčka z okrajů po složení navzájem sousední

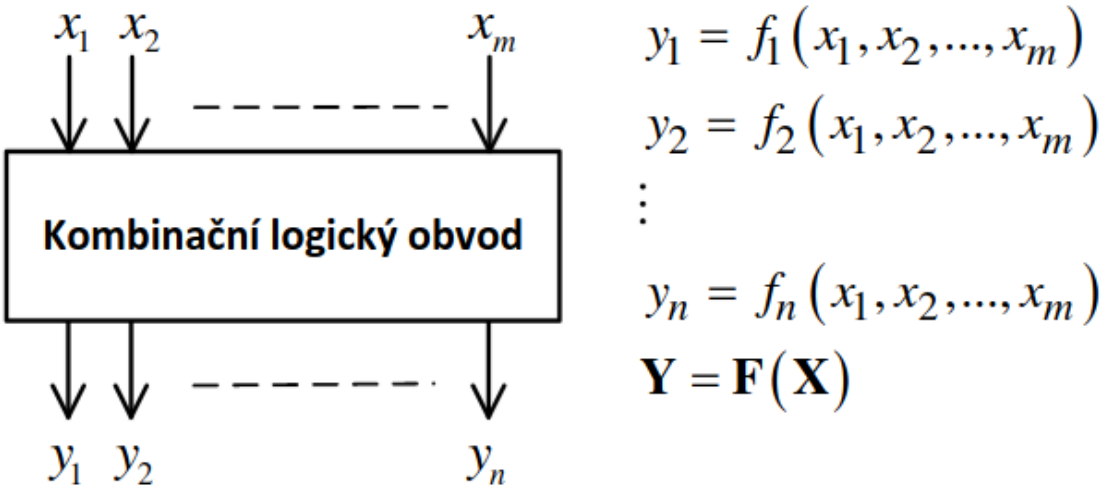


Karnaughovy mapy – zápis funkce pomocí mapy

napíšeme “1” (nebo čáru) do políčka s indexem, ve kterém nabývá funkce logické 1, “X” do políčka, ve kterém je hodnota funkce neurčitý stav a “0” do políčka s indexem, ve kterém dává funkce hodnotu logická 0



Kombinační logický obvod (KLO)



obecně logický obvod složený z log. hradel a dalších elementárních prvků, jehož výstupy y1, y2,. . . , yn jsou jednoznačně určeny kombinací vstupních hodnot x1, x2, . . . , xm

obvod neobsahuje žádný paměťový člen a žádnou zpětnou vazbu!

výstup obvodu y – je v každém okamžiku jednoznačně určen pouze vstupní kombinací!

pro m vstupů a n výstupů – můžeme sestavit pro popis obvodu 2n Booleových rovnic

ideální kombinační logický obvod – jakákoliv změna vstupní hodnoty vyvolá okamžitě změnu výstupní hodnoty obvodu – do další změny na vstupu je pak výstup konstantní

reálný (nedokonalý) kombinační logický obvod – vlivem doby zpoždění jednotlivých hradel obvodu se změna vstupu KLO projeví na jeho výstupu se zpožděním

doba zpoždění = časová prodleva mezi okamžikem změny na vstupu do nastavení správného výstupu obvodu (viz dále doba zpoždění hradel)

nenulová doba zpoždění logických hradel – vznik logických hazardů v obvodech

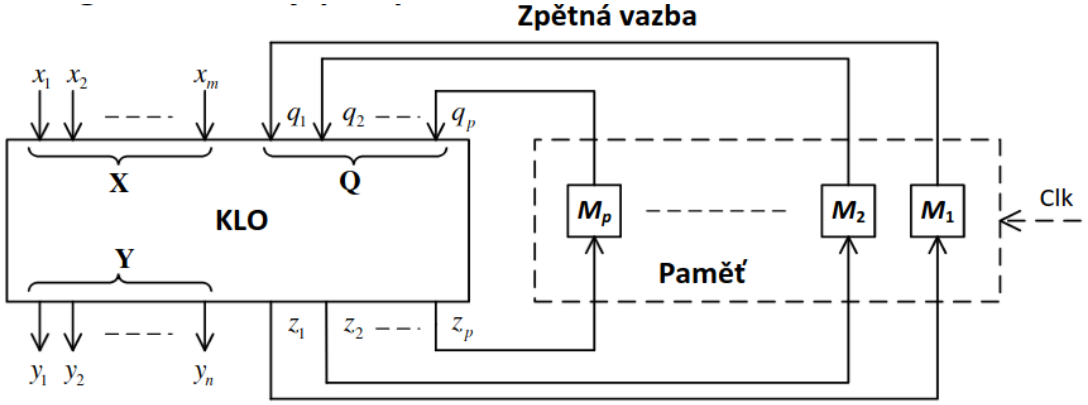
pro jejich popis a realizaci používáme zejména: 1. algebraické vyjádření – popis funkce obvodu s využitím zákonů Booleovy algebry 2. pravdivostní tabulka – tabulkové vyjmenování výstupů obvodu

pro vstupní kombinace 3. schéma obvodu – grafické vyjádření zapojení KLO pomocí elementárních logických hradel a obvodových prvků

typické příklady KLO

- jednoduché logické operace
- komparátory, paritní kodéry a dekodéry, přepínače
- multiplexory a demultiplexory
- převodníky kódů
- binární kodéry a dekodéry, prioritní kodéry, dekodéry segmentových displejů apod.
- sčítačky, odčítačky, násobičky, děličky

Sekvenční logické obvody (SLO)



sekvenční log. obvod (SLO) = kombinační log. obvod (KLO) + paměť + zpětná vazba

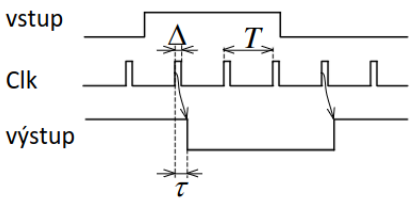
každý SLO je plně určen: 1. konečná množina vstupních stavů: $\mathbf{X} = \{x_1, x_2, \ldots, x_M\}$ 2. konečná množina výstupních stavů: $\mathbf{Y} = \{y_1, y_2, \ldots, y_N\}$ 3. konečná množina vnitřních stavů: $\mathbf{Q} = \{q_1, q_2, \ldots, q_P\}$ 4. konečná množina přechodových funkcí $\mathbf{G}$: $\mathbf{Q}^{t+1} = \mathbf{G}(\mathbf{X}^t; \mathbf{Q}^t) = g(x_1^t, x_2^t, \ldots, x_m^t; q_1^t, q_2^t, \ldots, q_p^t)$ 5. konečná množina výstupních funkcí F: - obvod typu Mealy $\mathbf{Y}^{t+1} = \mathbf{F}(\mathbf{X}^t; \mathbf{Q}^t) = f(x_1^t, x_2^t, \ldots, x_m^t; q_1^t, q_2^t, \ldots, q_p^t)$ - obvod typu Moore $\mathbf{Y}^{t+1} = \mathbf{F}(\mathbf{Q}^t) = f(q_1^t, q_2^t, \ldots, q_p^t)$

Mealy vs. Moore – synchronní režim

- **typ Mealy** – výstup obvodu závisí na aktuální hodnotě vstupů a vnitřních stavů:
- **typ Moore** – výstup obvodu závisí pouze na aktuální hodnotě vnitřních stavů:

u synchronního obvodu se vstupní hodnota Xt vyhodnocuje pouze během doby trvání hodinového impulzu Clk δ (v ideálním obvodu na jeho nekonečně strmou hranu)

ke změně vnitřního stavu obvodu dochází během doby T mezi dvěma po sobě následujícími impulzy hodinového signálu Clk

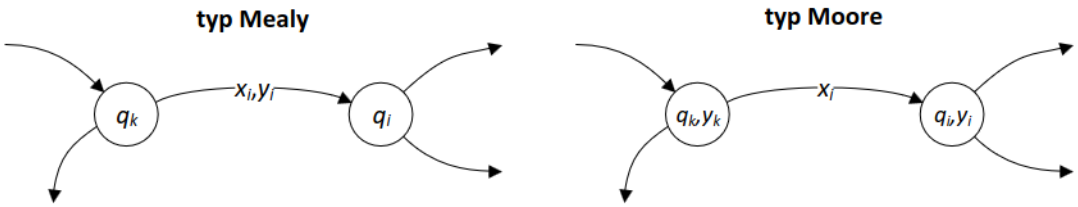


kde:

- Δ je doba trvání jednoho hodinového pulzu Clk
- T je jeho perioda
- τ představuje zpoždění výstupu obvodu

v případě asynchronního obvodu – změna vstupní hodnoty přímo způsobí změnu výstupní hodnoty se zpožděním τ

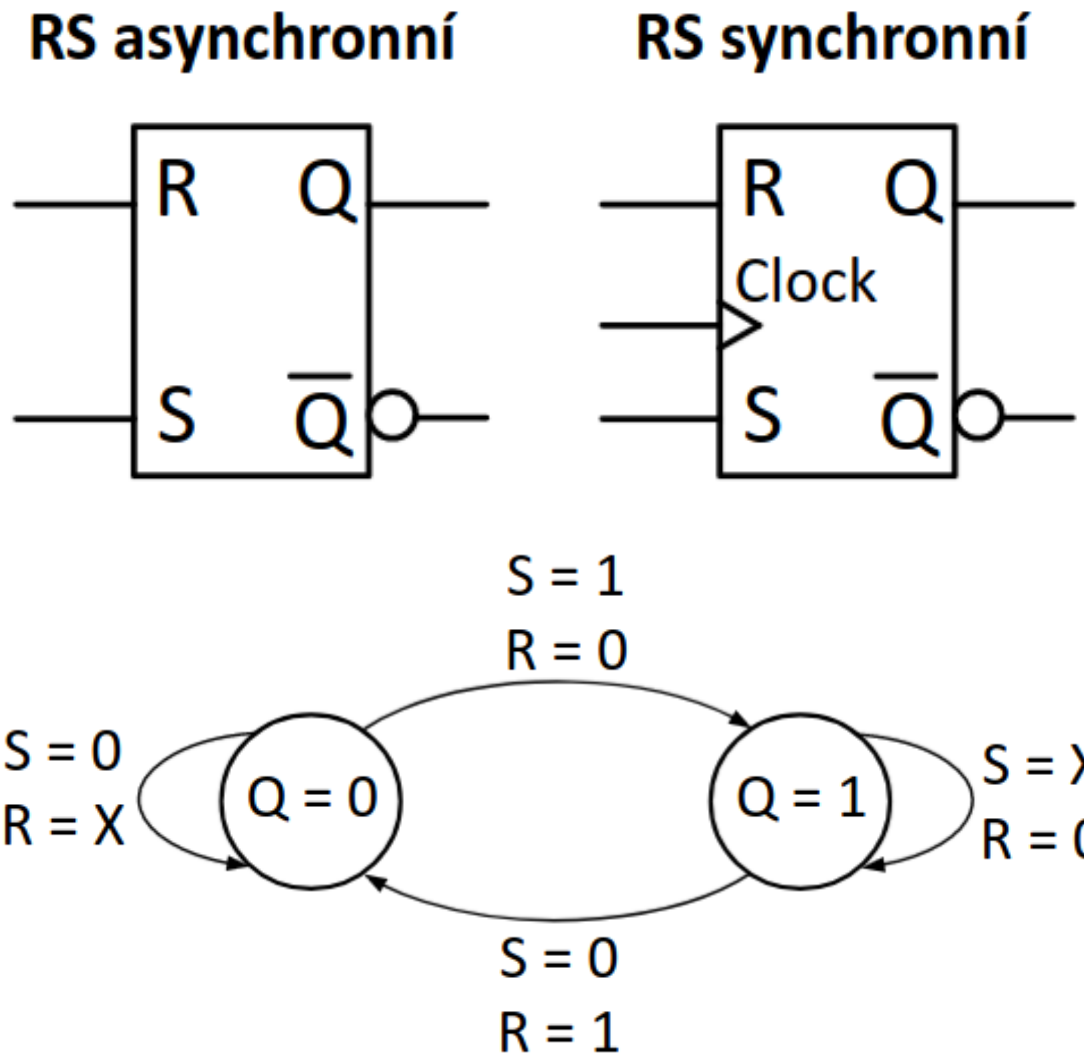
další změna vstupních hodnot tak musí přijít až po ukončení přechodu do nového vnitřního stavu – více se asynchronními obvody zabývat nebudeme



Klopný obvod RS (reset-set latch)

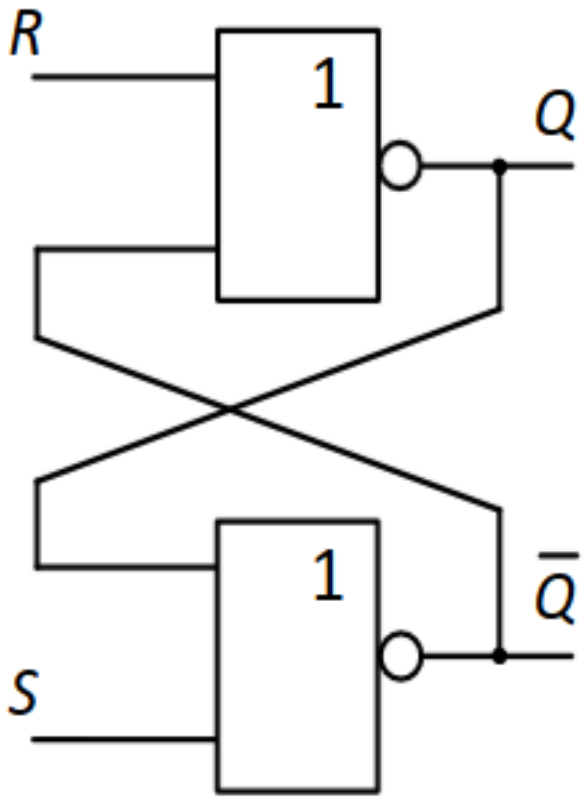
2 vstupy: S – Set, R – Reset, 2 výstupy: Q – přímý, Q – negovaný

asynchronní – hradla NOR, NAND, synchronní – řízený hladinou Clk

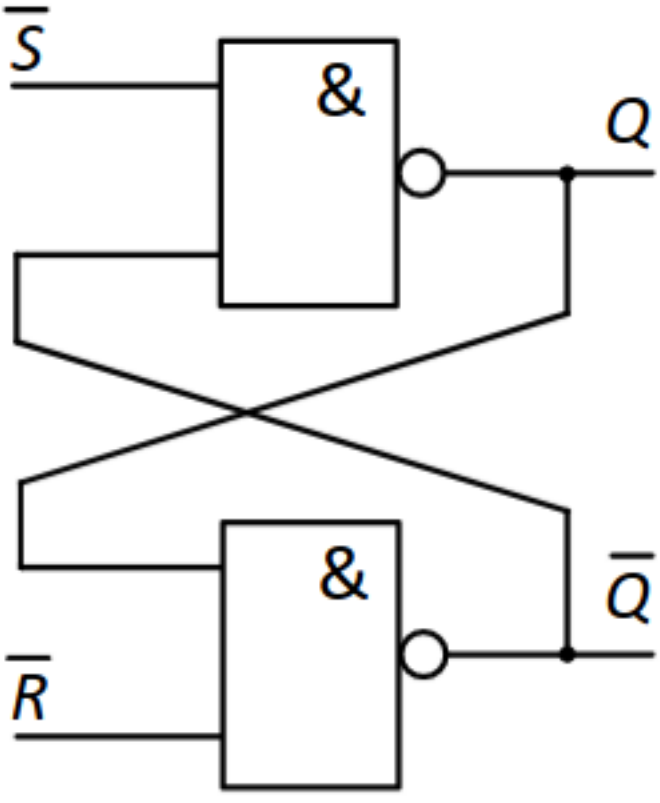


asynchronní

1. pokud S = logická 1 – Q = logická 1, Q = logická 0 -> jedničková transf.
2. pak S nastavíme na logickou 0 – výstupy se nezmění -> paměťová tr.
3. pak R nastavíme na logickou 1 – Q = logická 0, Q = logická 1 -> nulová tr.
4. pak R vrátíme na logickou 0 – výstupy se nezmění -> paměťová tr.
5. pokud S i R nastavíme na logickou 1 – výstup není určen – zakázaný stav!



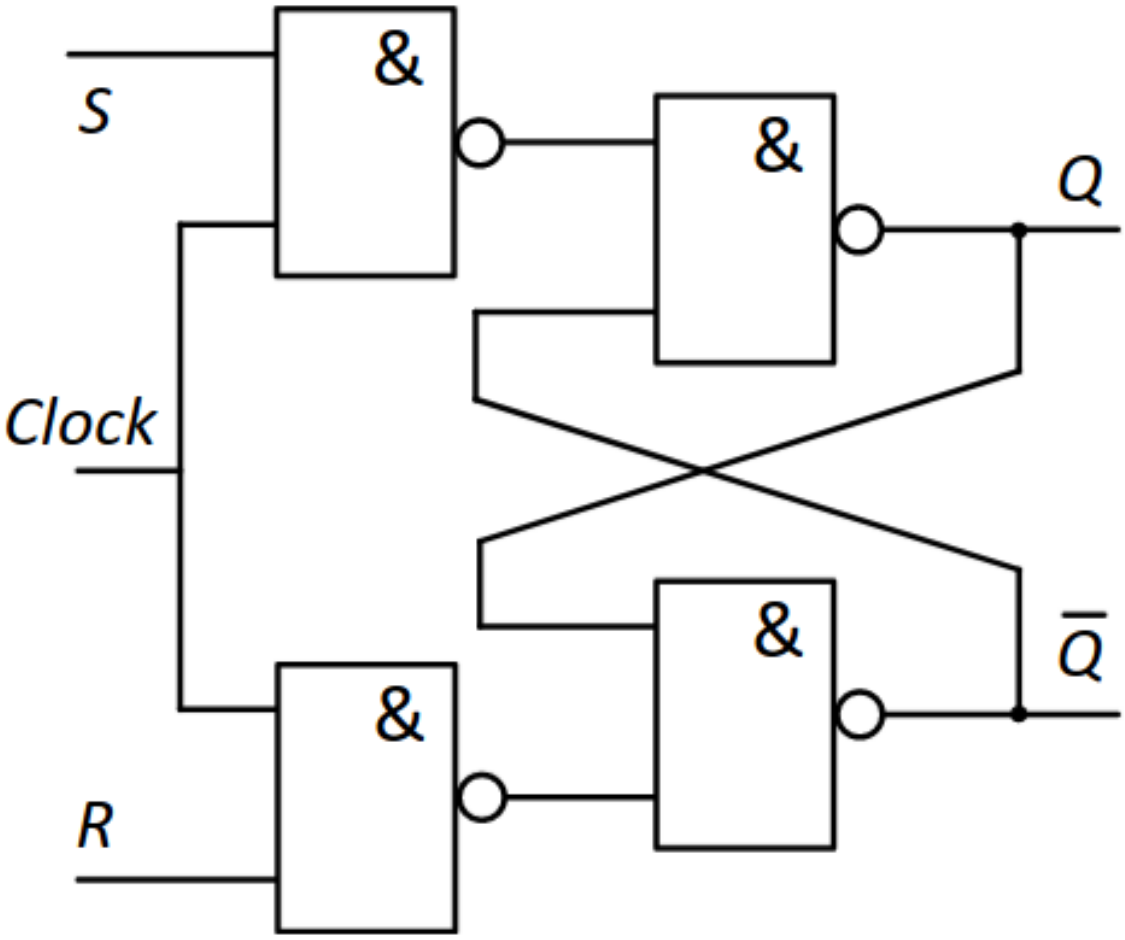
Obrázek 1: RS asynchronní – realizace NOR – aktivován logickou 1



Obrázek 2: RS asynchronní – realizace NAND – aktivován logickou 0

R	S	Q^{t+1}	transformace
0	0	Q^t	M
0	1	1	1
1	0	0	0
~~1~~	~~1~~	~~zakázaný stav~~	

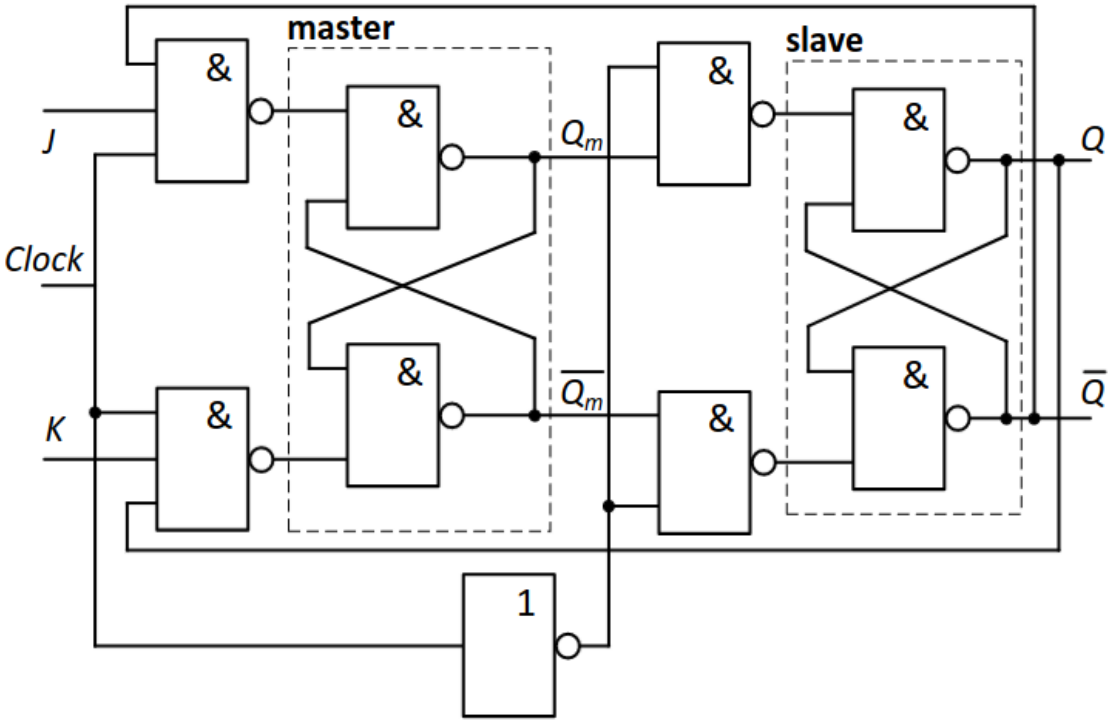
synchronní



Clock	R	S	Q^t	Q^{t+1}	Transformace
↑	0	0	0	0	Paměťová (M)
↑	0	0	1	1	
↑	0	1	0	1	Jedničková (1)
↑	0	1	1	1	
↑	1	0	0	0	Nulová (0)
↑	1	0	1	0	
↑	1	1	0	!	zakázaný stav
↑	1	1	1	!	
jiný	x	x	x	Q^t	-

Klopný obvod JK

problém základního JK – možnost vícenásobného kmitání (tzv. race-hazard). Problém způsoben dvojicí zpětných vazeb -> díky zpětným vazbám se změna výstupu obvodu dostane zpět na jeho vstup -> pokud stále trvá hrana (či hladina) vstupu Clock, způsobí to další překlopení obvodu a tak dále... -> ve výsledku se výstup obvodu může nekontrolovatelně rozkmitat mezi logickou 1 a 0 - první řešení – zvýšit frekvenci hodinového signálu Clock (zkrátit aktivní dobu obvodu) – ne vždy možné s ohledem na aplikaci - druhé řešení – tzv. master-slave zapojení



část master – reaguje na vzestupnou hranu Clock, část slave – sestupná hrana Clock

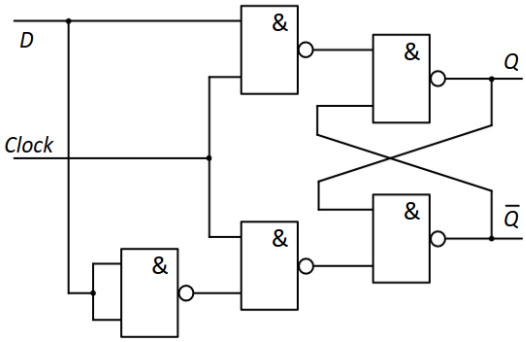
při vzestupné hraně Clock: - část master aktivní, na výstupech Qm a Qm se objeví příslušný výstup - vstup části slave však blokován (negace Clock) – slave část neaktivní - zpětné vazby zablokovány (slave část neaktivní)

při sestupné hraně Clock: - část master neaktivní – na výstupu zůstávají předchozí stavy Qm a Qm - část slave aktivní – zareaguje na vstupy Qm a Qm - zpětné vazby sice přenesou změnu zpět na vstup části master – ale ten je neaktivní

Klopný obvod D

synchronní klopný obvod RS, kdy spojíme vstupy R = S , jen jedničková a nulová transf.

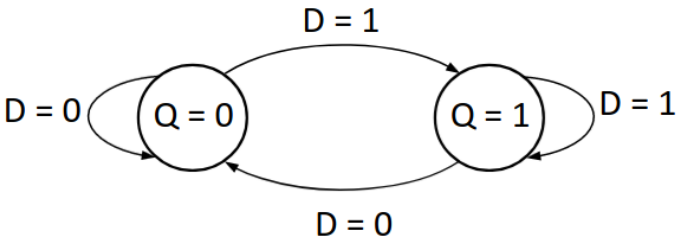
1 vstup D – data (delay), obvod D – paměťový člen s omezenou dobou záznamu



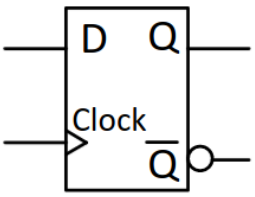
Clock	D	Q^t	Q^{t+1}	Transformace
↑	0	0	0	Nulová (0)
↑	0	1	0	
↑	1	0	1	Jedničková (1)
↑	1	1	1	
jiný	x	x	Q^t	-

obvod D = kopíruje hodnotu ze vstupu D na výstup Q (a negovanou hodnotu na výstup Q) s každou hranou (hladinou) hodinového vstupu Clock

dočasná paměť – hodnota ze vstupu D se uchová jen do následujícího pulzu Clock



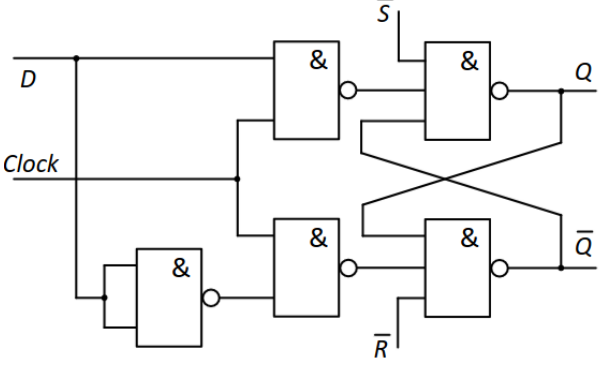
značka klopný obvod D



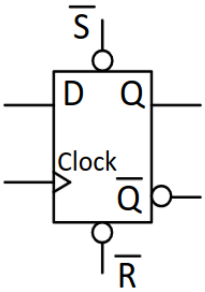
problém hladinového obvodu D – pokud je doba trvání hladiny Clock > doba trvání jedné hodnoty na vstupu D, tzn. pokud během aktivní doby se změní hodnota vstupu D vícekrát, změní se vícekrát i výstup a po skončení zůstane uložena jen poslední hodnota ze vstupu (tzv. ripple-through) - řešení – hranový klopný obvod D – použitím dodatečných hradel (obvykle NAND) a využitím jejich doby zpoždění zkrátíme trvání Clock – jen krátké pulsy (hrany)

s asynchronními vstupy R (clear) a S (preset)

dva asynchronní vstupy S, R jako pozůstatek původního klopného obvodu RS



klopný obvod D se vstupy S a R



vstup D je synchronní – je synchronizován pomocí hodinového vstupu Clock

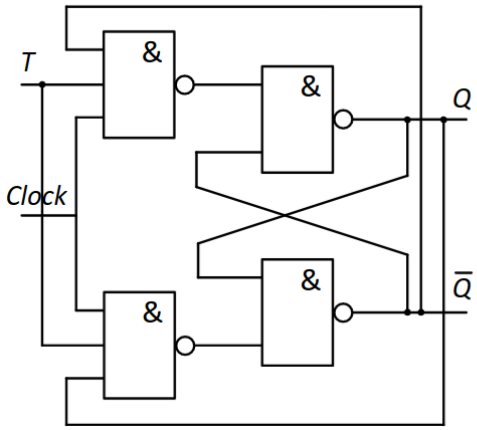
vstupy S a R jsou asynchronní – oba jsou aktivní v logické 0 a nejsou synchronizovány hodinovým vstupem Clock! – jejich funkce převáží (má prioritu) nad vstupem D! – proč? podívejte se do pravdivostní tabulky hradla NAND

Klopný obvod T

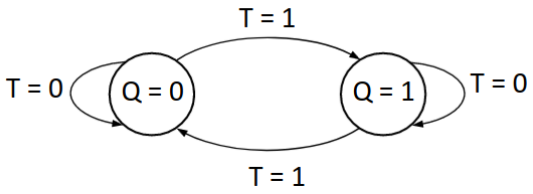
vychází z klopného obvodu JK, pouze spojíme dohromady oba vstupy J = K

1 vstup – T, jen transformace paměťová a klopná, T = toggle (překlápění)

klopný obvod T buď jen zaznamenává poslední hodnotu, nebo ji překlápí



Clock	T	Q^t	Q^{t+1}	Transformace
↑	0	0	0	Paměťová (M)
↑	0	1	1	
↑	1	0	1	Klopná (K)
↑	1	1	0	
jiný	x	x	Q^t	-



klopný obvod T značka

